

K-d Trees

(Secondary Key Retrieval)

Usage

- K-d tree (K dimensional tree) combines both primary and secondary key retrieval into a single structure.
- Used for range searches

Example:

- find all employees who earn more than 4000 TL,
or
- find all employees who earn more than 4000 TL
and have more than 2 children.

K-d Tree

- Allows to search over multiple dimensional data
 - Range queries in databases of multiple keys:
Ex. find persons with
 $34 \leq \text{age} \leq 49$ and $\$100k \leq \text{annual income} \leq \$150k$
 - GIS (geographic information system)
 - Computer graphics
- Extending BST from one dimensional to k-dimensional
 - It is a binary tree
 - Organized by levels (root is at level 0, its children level 1, etc.)
 - Tree branching at level 0 according to the first key, at level 1 according to the second key, etc.
- K-d Node
 - Each node has a vector of keys, in addition to the pointers to its subtrees.

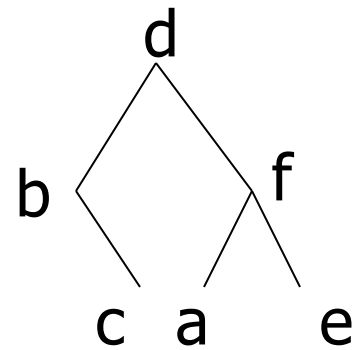
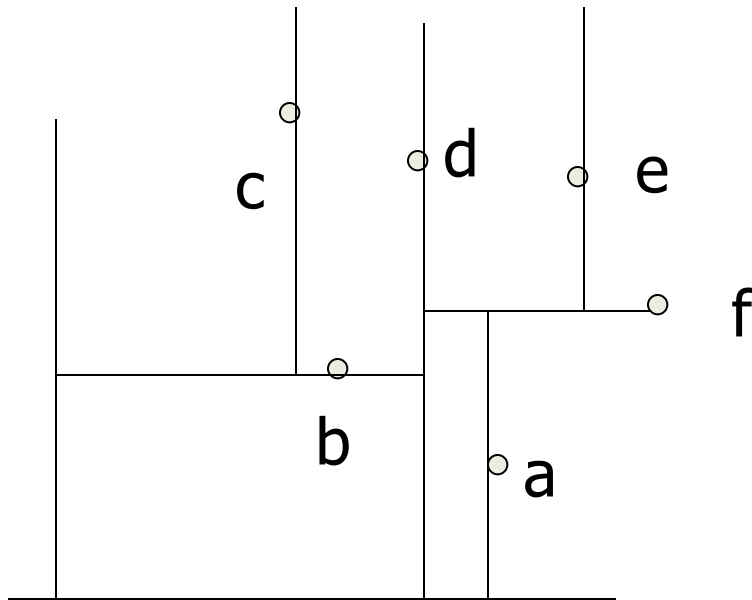
K-d tree definition

- A recursive space partitioning tree.
 - Partition along x , y , z , etc. axis in an alternating fashion.
 - Each internal node stores the splitting node along x (or y or z or ...).

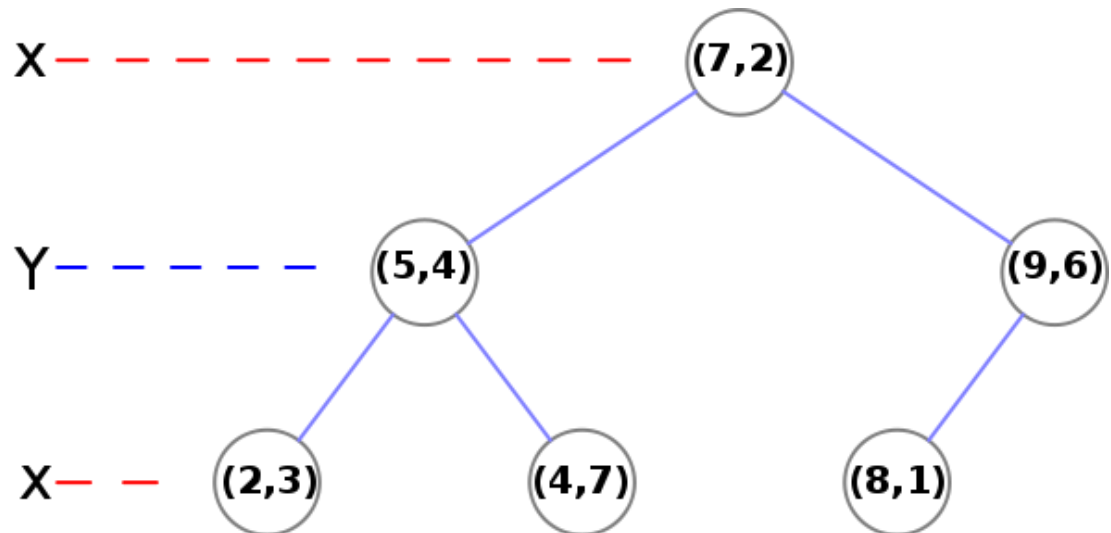
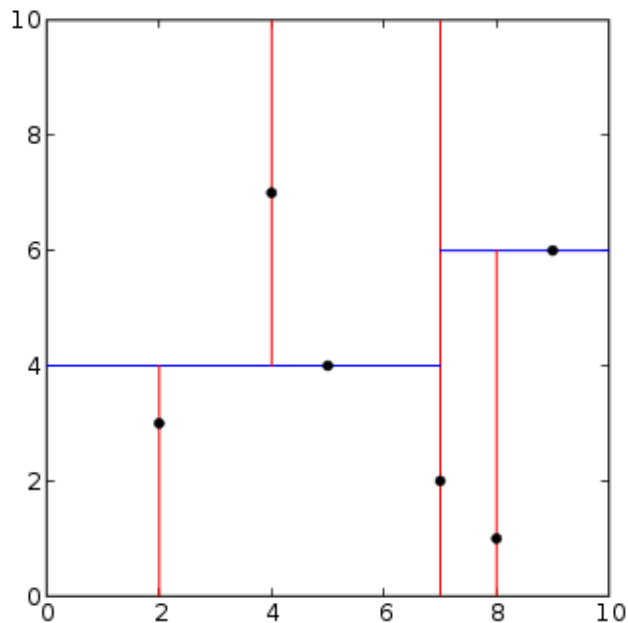
K-d tree

- Used for point location and multiple database queries, k is the number of the attributes to perform the search
- Geometric interpretation – to perform search in 2D space – 2-d tree
- Search components (x, y) interchange!

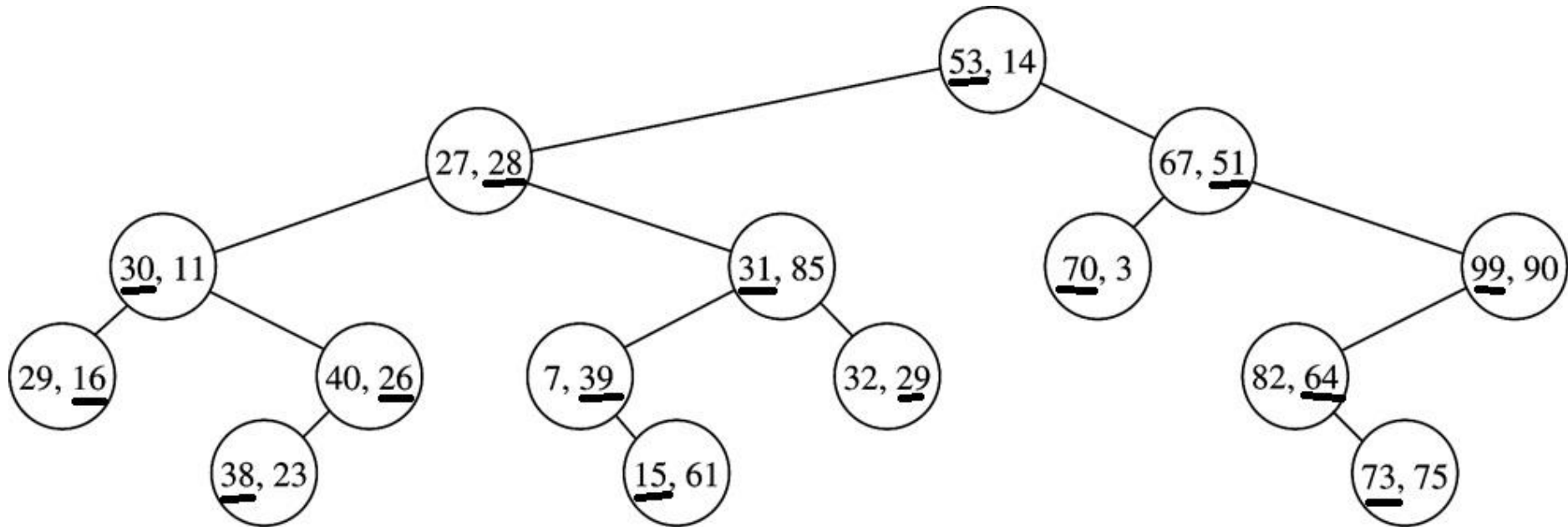
K-d tree example



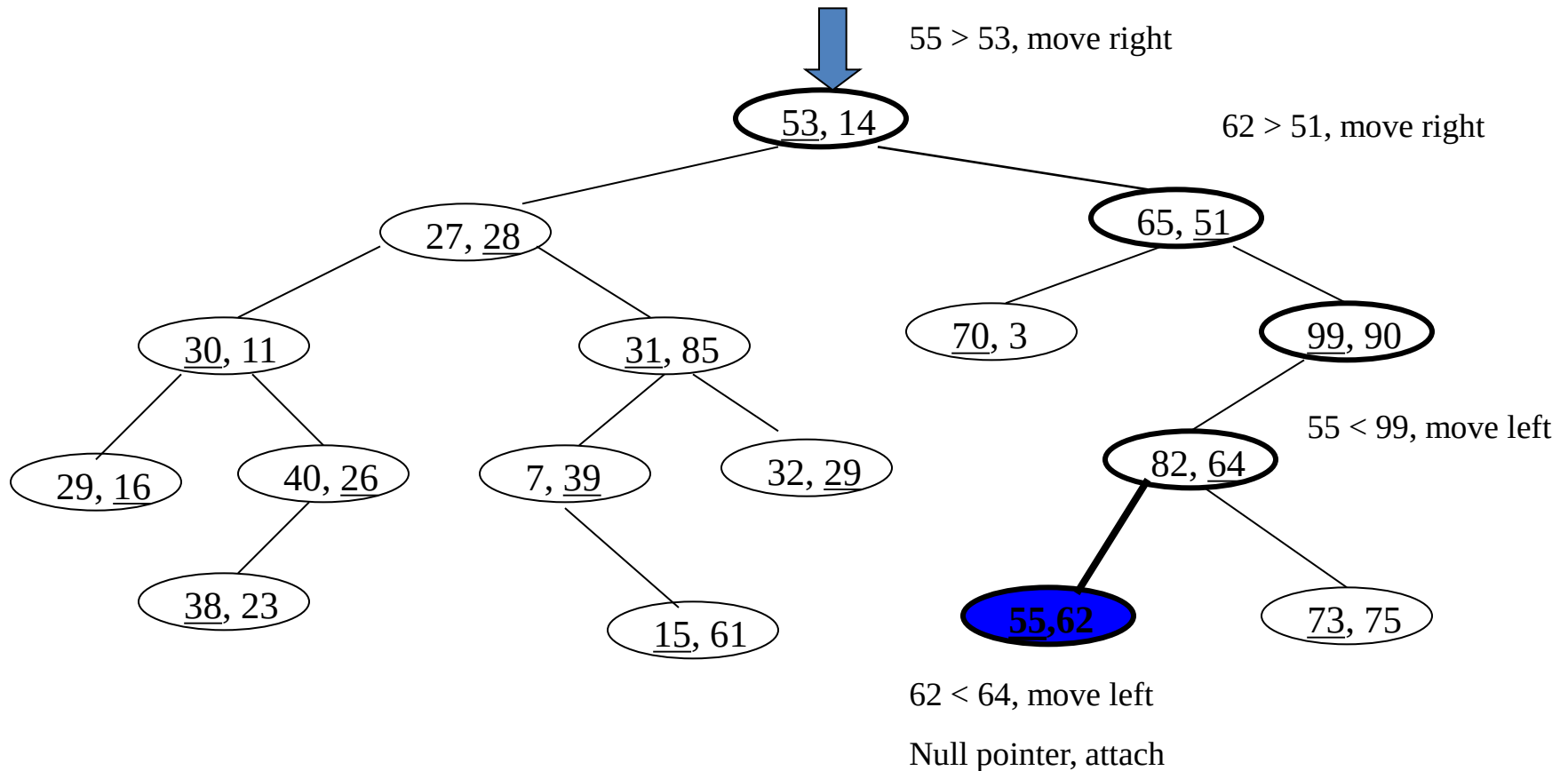
K-d tree example



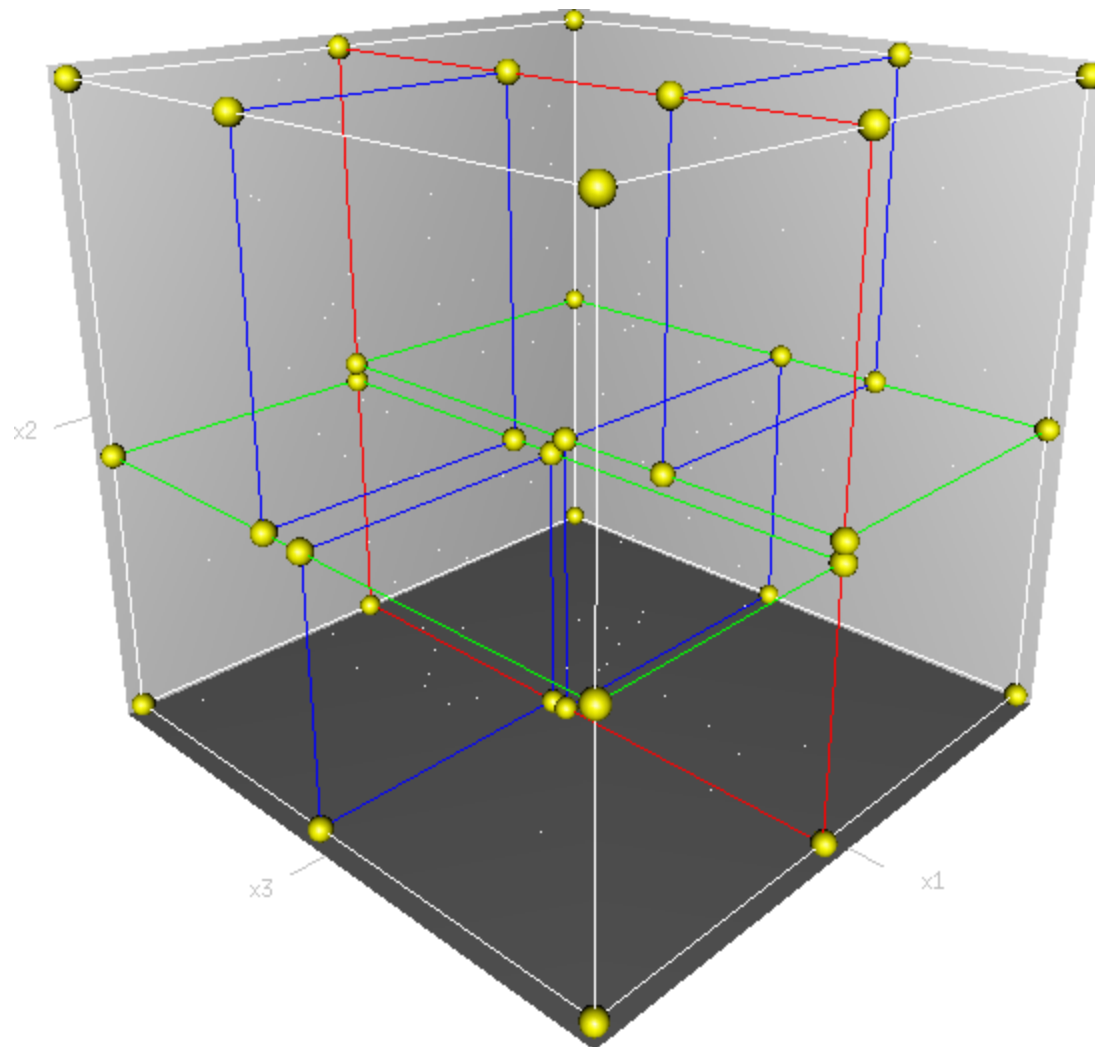
2-d tree example



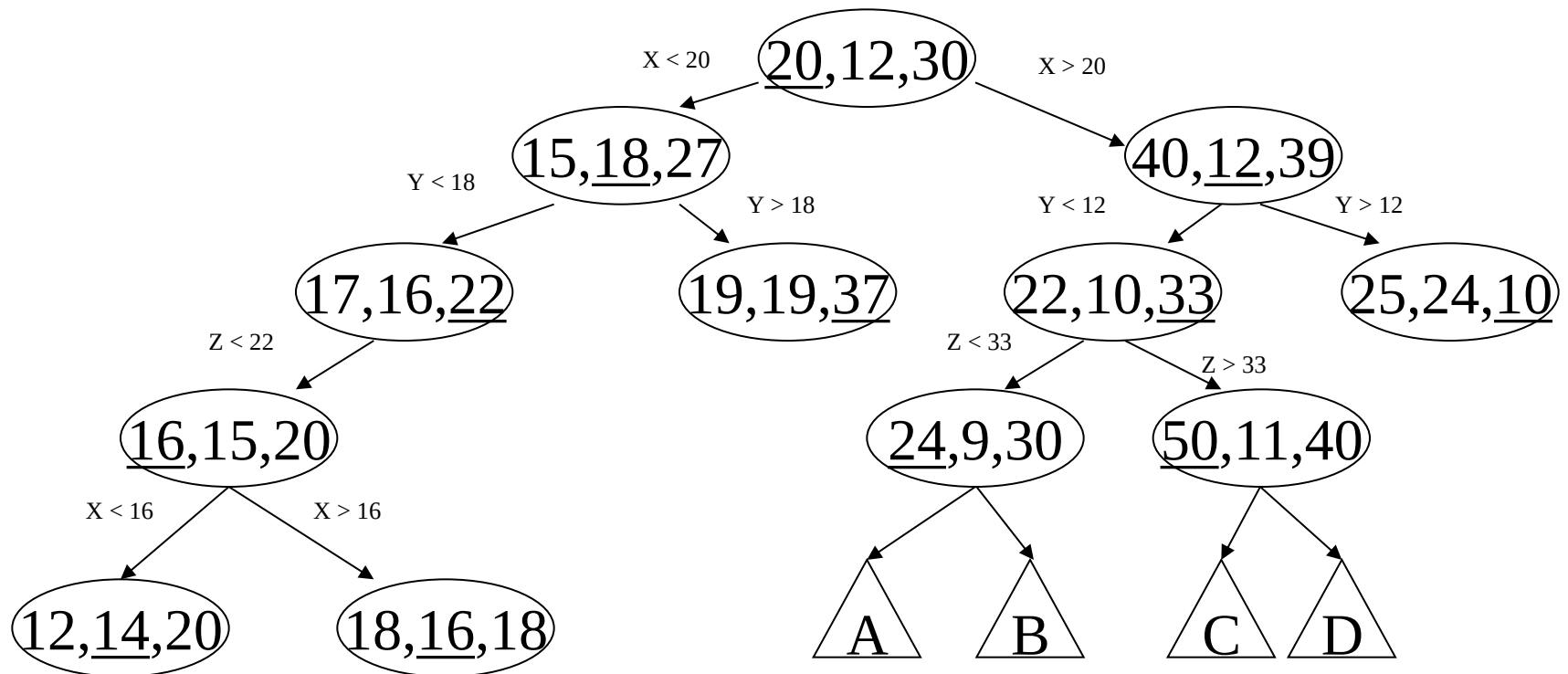
Insert (55, 62) into the following 2-D tree



3D K-d tree



3-D Tree example



What property (or properties) do the nodes in the subtrees labeled A, B, C, and D have?

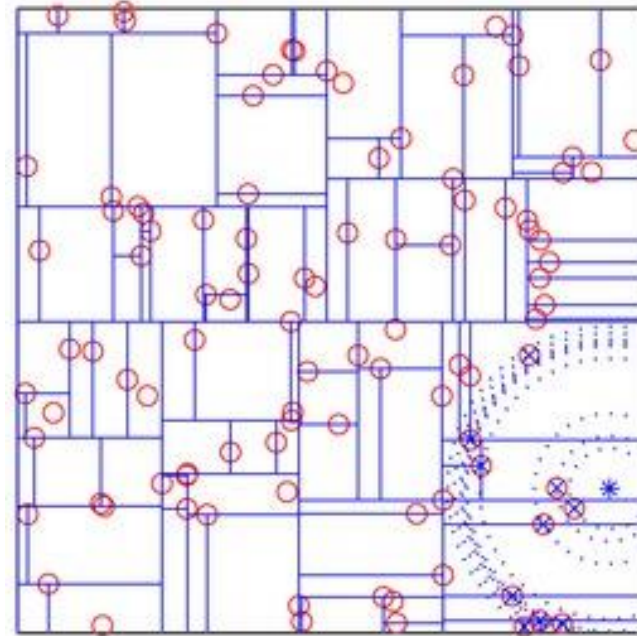
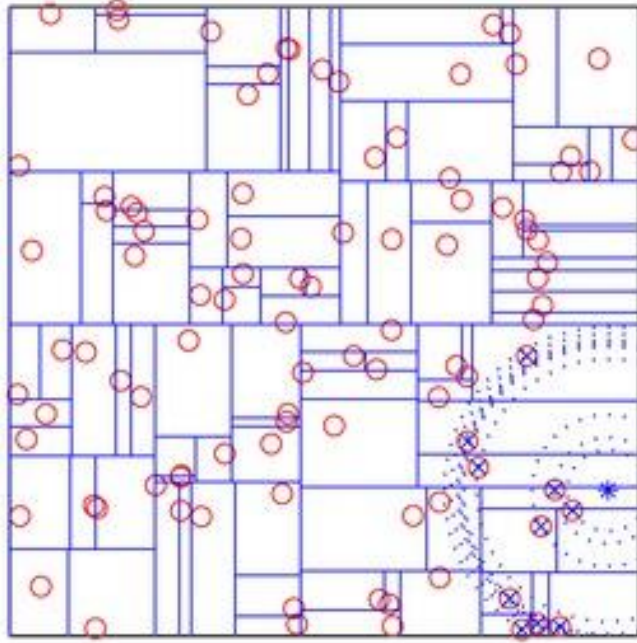
Construction

- ▶ The canonical method of k -d tree construction is the following:
- ▶ As one moves down the tree, one cycles through the axes used to select the splitting planes. (For example, the root would have an x -aligned plane, the root's children would both have y -aligned planes, the root's grandchildren would all have z -aligned planes, the next level would have an x -aligned plane, and so on, if there are 3 dimensions)
- ▶ Points are inserted by selecting the median of the points being put into the subtree, with respect to their coordinates in the axis being used to create the splitting plane. (Note the assumption that we feed the entire set of points into the algorithm up-front.)

Construction

- ▶ This method leads to a balanced k -d tree, in which each leaf node is about the same distance from the root. However, balanced trees are not necessarily optimal for all applications.
- ▶ Note also that it is not *required* to select the median point. In that case, the result is simply that there is no guarantee that the tree will be balanced. A simple heuristic to avoid coding a complex linear-time median-finding algorithm or using an $O(n \log n)$ sort is to use sort to find the median of a *fixed* number of *randomly* selected points to serve as the cut line

K-d tree – mean vs median



kd-tree partitions of a uniform set of data points, using the mean (left image) and the median (right image) thresholding options.

Median: The middle value of a set of values. *Mean*: The arithmetic average.

(Andrea Vivaldi and Brian Fulkersson)

<http://www.vlfeat.org/overview/kdtree.html>

Insertion

- ▶ One inserts a new point to a k -d tree in the same way as one adds an element to any other [search tree](#).
- ▶ First, traverse the tree, starting from the root and moving to either the left or the right child depending on whether the point to be inserted is on the "left" or "right" side of the splitting plane.
- ▶ Once you get to the node under which the child should be located, add the new point as either the left or right child of the leaf node, again depending on which side of the node's splitting plane contains the new node.
- ▶ Adding points in this manner can cause the tree to become unbalanced, leading to decreased tree performance.

Balancing

- Balancing a k -d tree requires care. Because k -d trees are sorted in multiple dimensions, the [tree rotation](#) technique cannot be used to balance them — this may break the invariant.
- Several variants of balanced k -d tree exists. They include divided k -d tree, pseudo k -d tree, K-D-B tree, h-B tree and Bk-d tree. Many of these variants are [adaptive k-d tree](#).

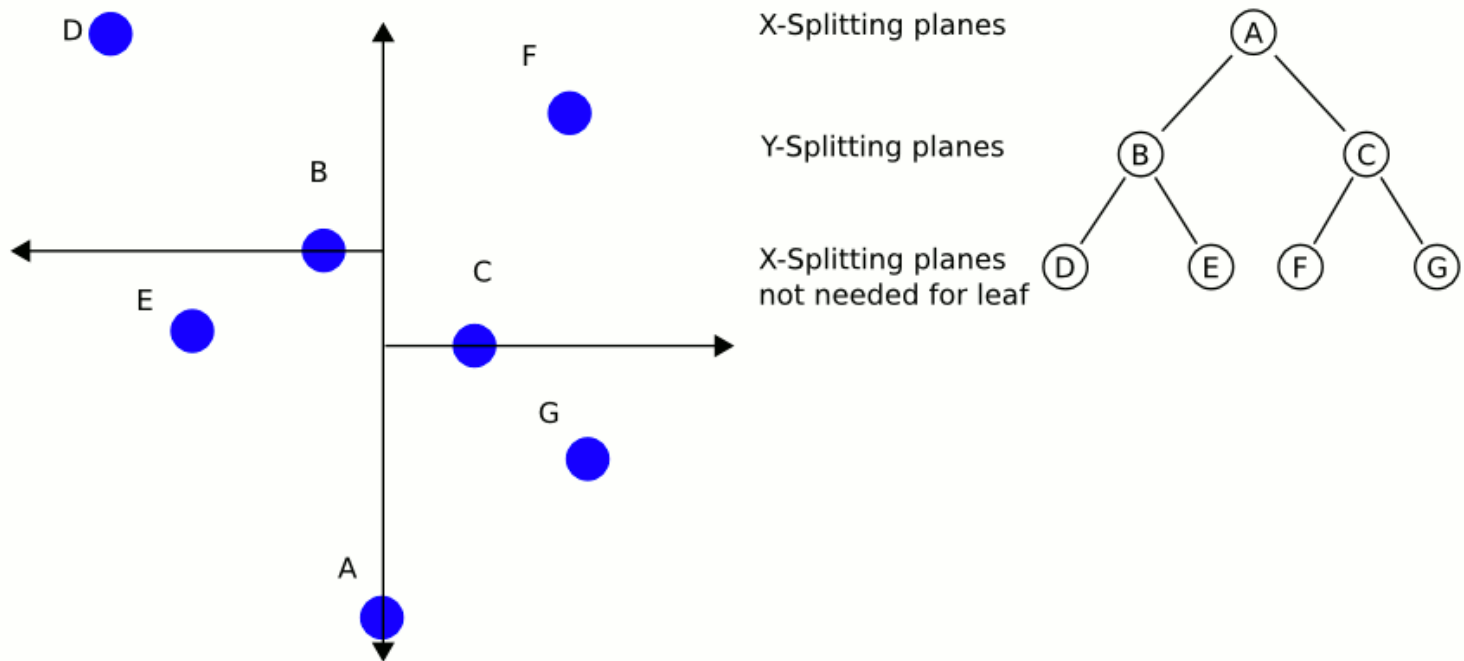
Quering

- ▶ K-d tree query uses a best-bin first search heuristic. This is a branch-and-bound technique that maintains an estimate of the smallest distance from the query point to any of the data points down all of the open paths.
- ▶ K-d tree query supports two important operations: *nearest-neighbor search* and *k-nearest neighbor search*. The first returns nearest-neighbor to a query point, the latter can be used to return the k nearest neighbors to a given query point Q .

Range search

- K-d tree provide convenient tool for range search query in databases with more than one key. The search might go down the root in both directions (left and right), but can be limited by strict inequality on key value at each tree level.
- K-d tree is one of the data structures that allow easy multi-key search.

K-d tree



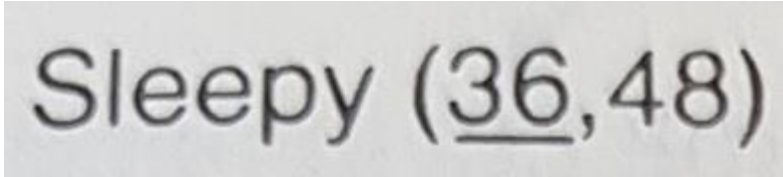
<http://upload.wikimedia.org/wikipedia/en/9/9c/KDTree-animation.gif>

Example

- Representative data records (Ms. White and 7 dwarfs)

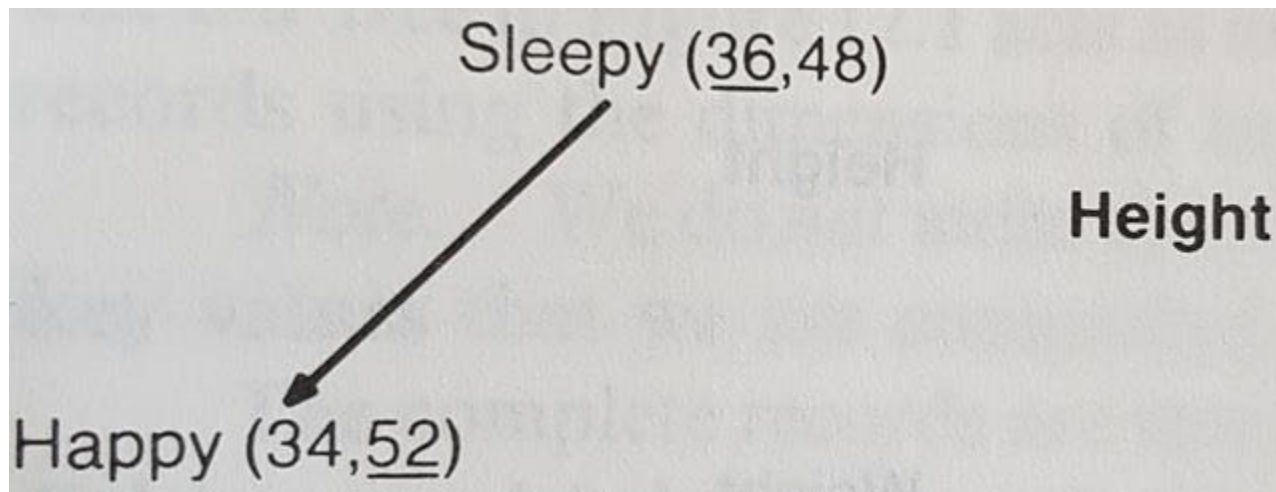
Name	Height	Weight	Other data
Sleepy	36	48	
Happy	34	52	
Doc	38	51	
Dopey	37	54	
Grumpy	32	55	
Sneezy	35	46	
Bashful	33	50	
Ms. White	62	98	

Insertion

- We insert each data record in the given order for this dataset.
- We use 2 attributes (i.e., height and weight), so the tree will be a 2-d tree
- First insert Sleepy(36, 48) starting with the height attribute. You can start with any one attribute.
- The root node is 

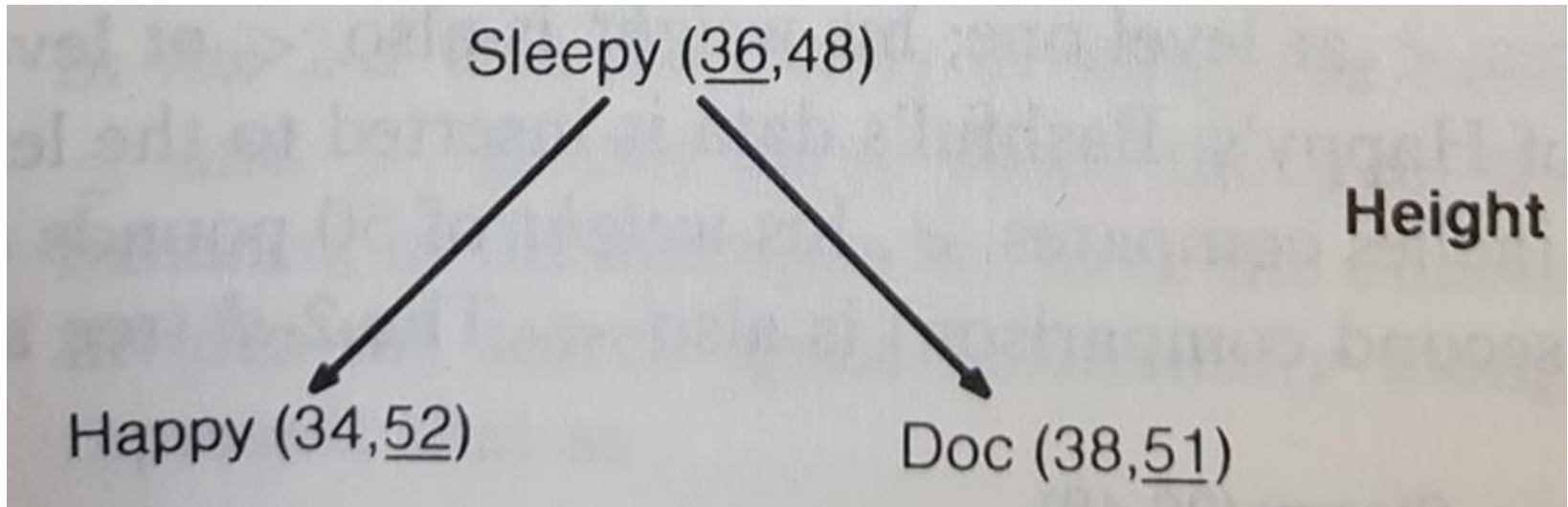
Insertion (Cont'd)

- Insert Happy(34, 52)



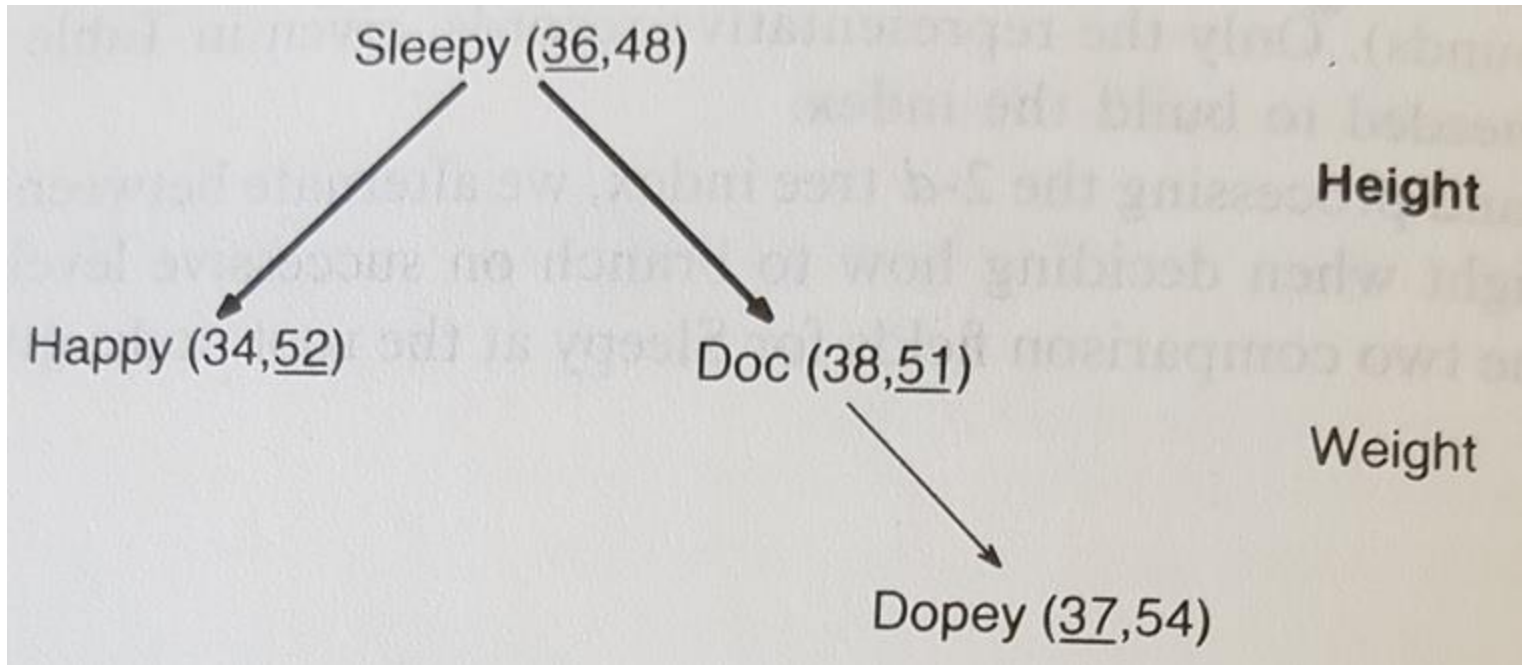
Insertion (Cont'd)

- Insert Doc(38, 51)



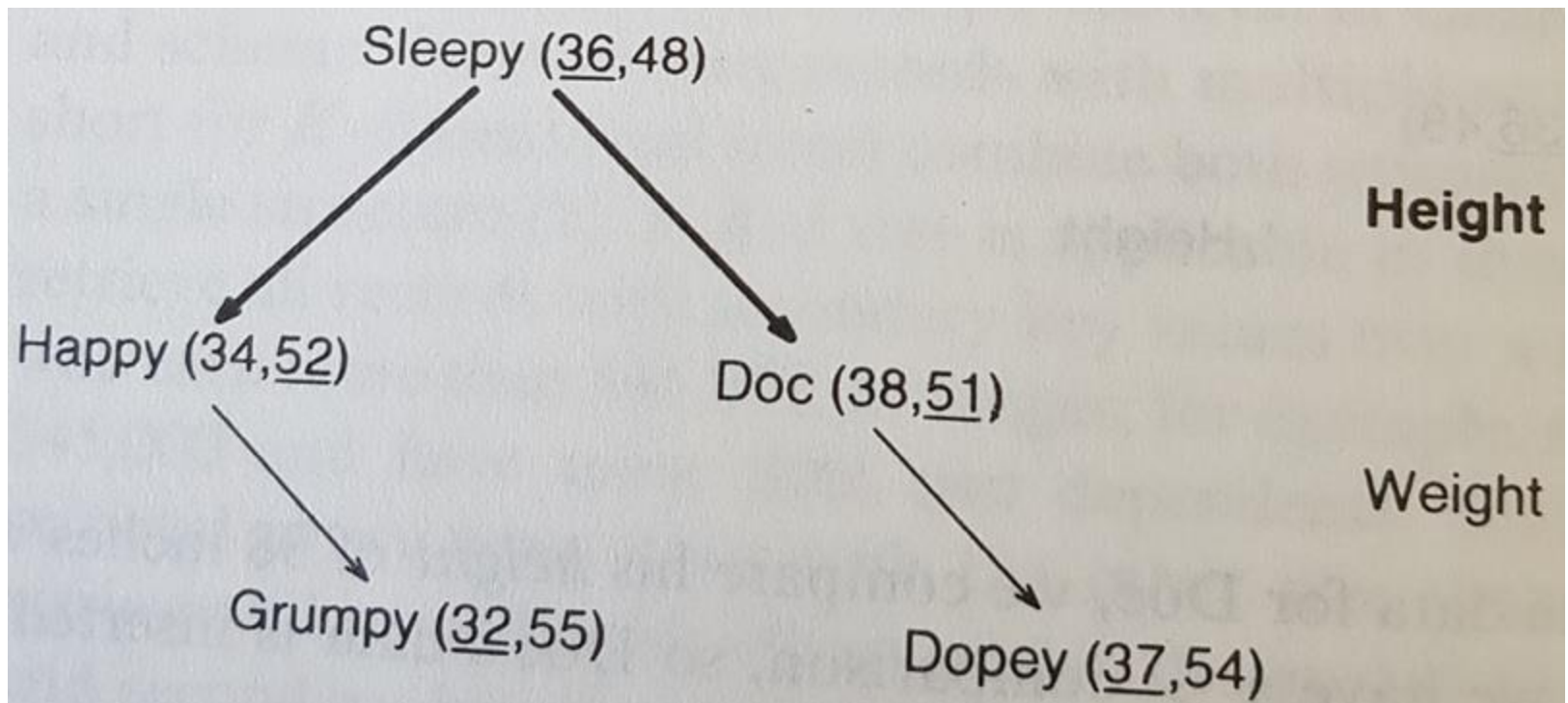
Insertion (Cont'd)

- Insert Dopey(37, 54)



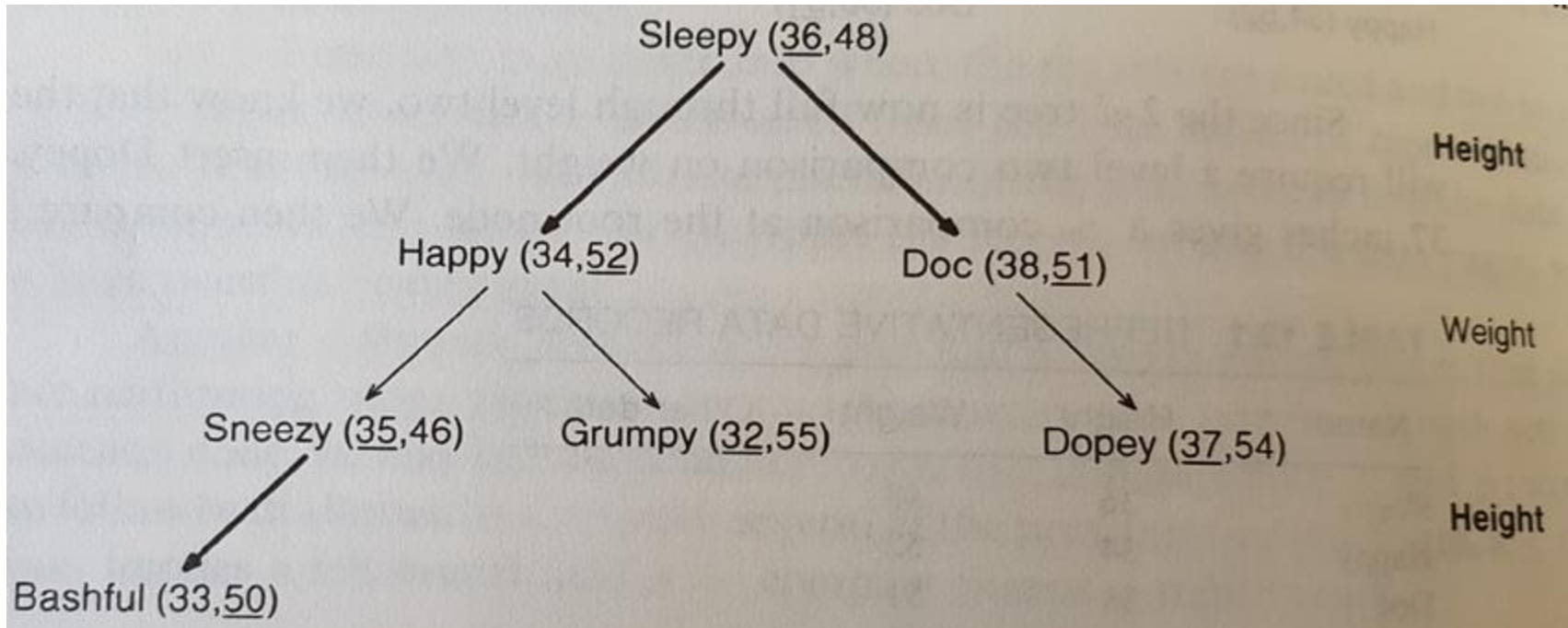
Insertion (Cont'd)

- Insert Grumpy(32, 55)



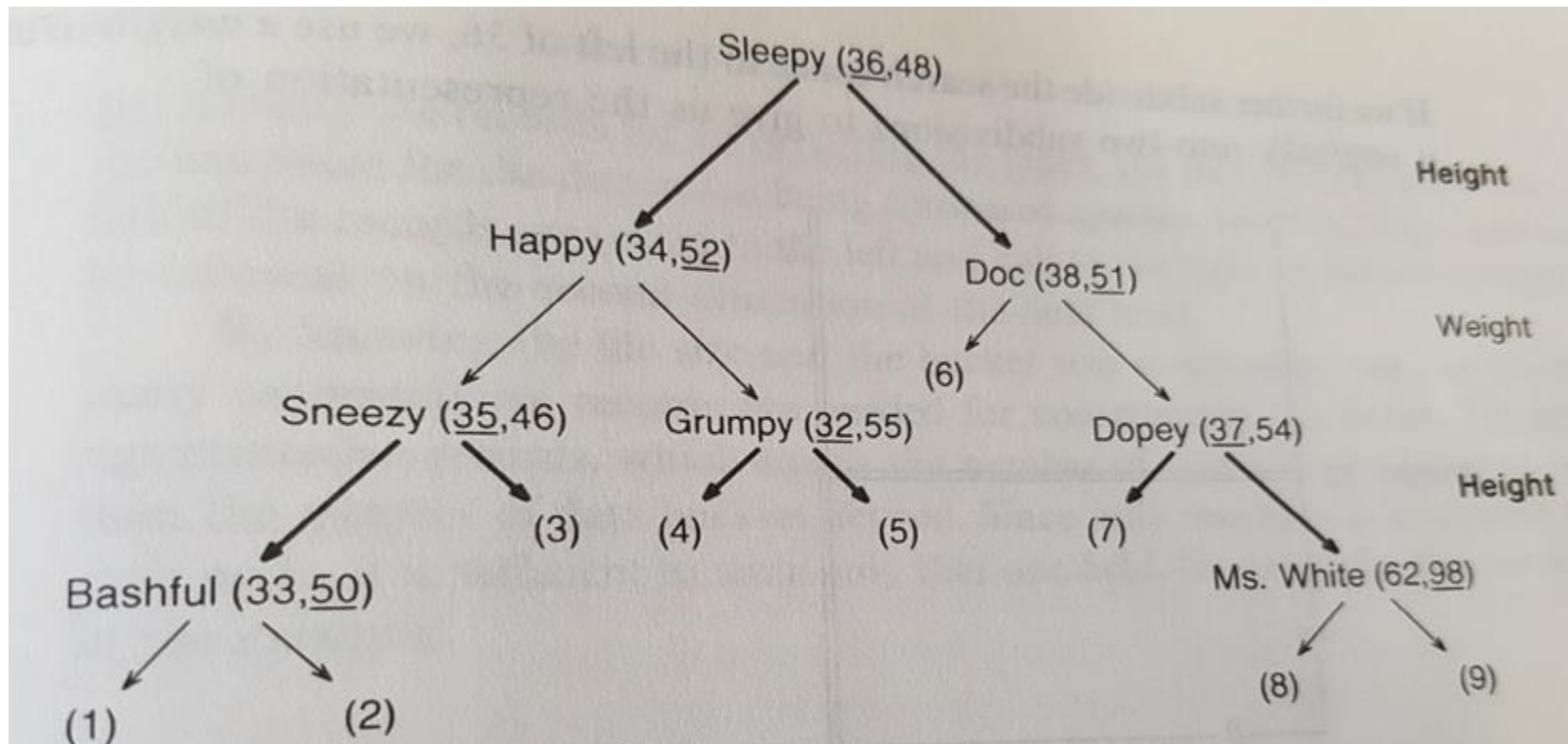
Insertion (Cont'd)

- Insert Sneezy(35, 46), then Bashful(33,50)



Insertion (Cont'd)

- Insert Ms. White(62, 98)



Complexity

- ▶ Building a static k -d tree from n points takes $\underline{O}(n \log^2 n)$ time if an $\underline{O}(n \log n)$ sort is used to compute the median at each level.
- ▶ The complexity is $\underline{O}(n \log n)$ if a linear [median-finding](#) algorithm is used.
- ▶ Inserting a new point into a balanced k -d tree takes $O(\log n)$ time.
- ▶ Removing a point from a balanced k -d tree takes $O(\log n)$ time.
- ▶ Querying an axis-parallel range in a balanced k -d tree takes $O(n^{1-1/k} + m)$ time, where m is the number of the reported points, and k the dimension of the k -d tree.

Applications

- Query processing in sensor networks
- Nearest-neighbor searchers
- Optimization
- Ray tracing
- Database search by multiple keys